

Analisi e Progettazione del Software

Emanuele Montani
Università degli Studi di Milano-Bicocca

Contents

0	Panoramica del Corso	5
0.1	Organizzazione	5
0.2	Materiali	5
0.3	Argomenti	5
0.4	Esame	5
1	Introduzione	6
1.1	Che cos'è il Software?	6
1.2	Ingegneria del Software	6
1.3	Il Processo di Sviluppo Software	6
1.4	CASE: Computer-Aided Software Engineering	7
1.5	Attributi di un Buon Software	7
1.6	Responsabilità Etica e Professionale	7
2	Analisi dei Requisiti	8
2.1	Ingegneria dei Requisiti	8
2.2	Processi Software	8
2.2.1	Modello a Cascata	8
2.2.2	Sviluppo Iterativo	9
2.3	Agile	9
2.3.1	Scrum	9
2.3.2	Unified Process (UP)	10
2.4	Analisi Orientata agli Oggetti (OOA)	10
3	UML: Casi d'Uso	11
3.1	Definizione	11
3.2	Struttura	11
3.3	Attore Primario e Obiettivo	11
3.4	Qualità delle Estensioni	11
3.5	I Tre Test per Validare i Casi d'Uso	11
3.5.1	Test del Capo	11
3.5.2	Test EBP (Elementary Business Process)	12
3.5.3	Test della Dimensione	12
3.6	Diagramma dei Casi d'Uso	13
3.6.1	Attore	13
3.6.2	Caso d'Uso	13

3.6.3	Associazione	13
3.6.4	System Boundary	13
3.6.5	Include	13
3.6.6	Extend	13
3.6.7	Generalization	13
3.7	Modello di Dominio	13
3.8	Diagrammi di Sequenza di Sistema (SSD)	14
3.9	Contratti delle Operazioni	14
3.10	Tracciabilità: Casi d'Uso → SSD → Contratti	15
3.11	Template pratico di contratto	15
4	Diagrammi di Dominio	16
4.1	Struttura di una Classe di Dominio	16
4.2	Notazione dei Diagrammi di Classe	16
4.2.1	Classe	16
4.2.2	Associazione	16
4.2.3	Associazione Riflessiva	16
4.2.4	Aggregazione	16
4.2.5	Composizione	16
4.2.6	Generalizzazione	16
4.2.7	Dipendenza	16
4.2.8	Interfaccia	16
4.3	Vincoli sulle Generalizzazioni	17
4.4	Classi Astratte e Polimorfismo	17
4.5	Aggregazione vs. Ereditarietà	17
5	Diagrammi di Sequenza	18
5.1	Elementi Principali	18
5.2	Notazione dei Diagrammi di Sequenza	18
5.2.1	Messaggio Sincrono	18
5.2.2	Messaggio Asincrono	18
5.2.3	Creazione Oggetto	18
5.2.4	Frammento loop	18
5.2.5	Frammento alt	18
5.2.6	Frammento opt	18
5.3	Da SSD a Diagrammi di Progetto	18
5.4	Oggetti attivi e sincrónico/asincrono	19
6	Architettura Logica	20
6.1	Strati (Layers)	20
6.2	Architettura Stretta vs. Rilassata	20
6.2.1	Architettura Stretta	20
6.2.2	Architettura Rilassata	20
6.3	Architettura Tipica a Quattro Strati	20
6.4	Partizioni	21
6.5	Diagrammi dei Package UML	21
6.6	Benefici dell'Architettura a Strati	21
6.7	Principio di Separazione Modello-Vista	21

7	Verso la progettazione a oggetti	22
7.1	Tre approcci pratici	22
7.2	Agile Modeling	22
7.3	Modelli statici e dinamici	22
8	Diagrammi di Interazione	23
8.1	Sequenza vs. Comunicazione	23
8.2	Messaggi e responsabilità	23
8.3	Frammenti combinati più usati	23
8.4	Relazione con i diagrammi di classe	23
8.5	Notazione compatta (come disegnarli)	24
8.5.1	Comunicazione	24
8.5.2	Messaggio condizionale	24
8.5.3	Numerazione annidata	24
8.5.4	Messaggio iterativo	24
8.6	Checklist di coerenza	24
9	Diagrammi di Classe di Progetto	25
9.1	Contenuto della classe UML	25
9.2	Dal modello di dominio al design	25
9.3	Relazioni e raffinamenti tipici	25
9.4	Classi e operazioni astratte	25
9.5	Notazione compatta (come disegnarli)	26
9.5.1	Classe UML	26
9.5.2	Associazione unidirezionale	26
9.5.3	Classe astratta	26
9.5.4	Dipendenza	26
10	Diagrammi di Attività	27
10.1	Elementi principali	27
10.2	Controllo del flusso	27
10.3	Uso consigliato	27
10.4	Notazione compatta (come disegnarli)	27
10.4.1	Flusso base	27
10.4.2	Decision + Merge	27
10.4.3	Fork + Join	27
10.4.4	Swimlane	27
10.5	Regole operative importanti	28
11	Diagrammi di Stato (Statechart)	29
11.1	Concetti base	29
11.2	Quando usarli	29
11.3	Stati composti	29
11.4	Linee guida pratiche	29
11.5	Notazione compatta (come disegnarli)	30
11.5.1	Stati e transizioni	30
11.5.2	Transizione con guardia	30
11.5.3	Stato composto	30
11.5.4	Stato finale	30

11.6	Schemi visuali avanzati	30
11.6.1	Stato ortogonale	30
11.6.2	Comunicazione asincrona	30
11.6.3	History semplice H	30
11.6.4	History multilivello H*	30
12	Principi GRASP	30
12.1	Creator	30
12.2	Information Expert (Esperto)	31
12.3	Low Coupling (Accoppiamento Basso)	31
12.4	High Cohesion (Alta Coesione)	31
12.5	Controller	31
12.6	Polimorfismo	31
12.7	Protected Variazioni	31
12.8	Pure Fabrication (Fabbricazione Pura)	32
12.9	Indirezione	32
12.10	Mappa visuale dei principi GRASP	32
13	Design Patterns	32
13.1	Mappa visuale: scopo dei pattern	32
13.2	Combinazioni frequenti (visione pratica)	33
13.3	Creational Patterns	33
13.4	Structural Patterns	33
13.5	Behavioral Patterns	34
14	Dalla progettazione al codice	35
14.1	Progettare la visibilità	35
14.2	Assegnazione delle responsabilità e tipi di visibilità	35
14.3	TDD: Test-Driven Development	36
14.4	Refactoring e Code Smells	36
14.4.1	Code Smells principali	37
14.4.2	Tecniche di refactoring fondamentali	37
14.5	Dai diagrammi al codice: flusso di lavoro	38

0 Panoramica del Corso

0.1 Organizzazione

- **Docenti (Turno M/Z):** Lezioni – Francesca Arcelli Fontana; Esercitazioni – Claudia Raibulet; Laboratorio – Gabriella Nifosi e Claudia Raibulet.
- **Orario:** Mercoledì 10:30–13:30 (lezioni, U1-09); Venerdì 8:30–10:30 (esercitazioni, U3-05); Mercoledì 14:30–17:30 (laboratorio, U14, dal 18/03/2026).
- **Strumenti:** Visual Paradigm (diagrammi UML), Git (controllo versione).
- **Piattaforma:** elearning.unimib.it (Moodle) – registrarsi entro la prima settimana.

0.2 Materiali

- Sommerville, *Ingegneria del Software*, ultima edizione, Pearson.
- Larman, *Applicare UML e i Pattern*, ultima edizione, Pearson (**testo principale**).
- M. Fowler, *UML Distilled*, ultima edizione, Pearson.
- Lucidi, articoli e video disponibili su Moodle (non sostituiscono il libro di testo).

0.3 Argomenti

- Processi software e processo agile; sviluppo iterativo.
- Analisi dei Requisiti: casi d'uso, modellazione di dominio.
- Progettazione object-oriented (OOA/D) e principi di progettazione.
- Design Pattern.
- Sviluppo test-driven (TDD) e traduzione del progetto in codice (introduzione).
- Code Smell e Refactoring (introduzione).

0.4 Esame

- **Pre-Appello (I Appello):** estensione del progetto di laboratorio (gruppo, 36h) + prova orale obbligatoria. Pesi: orale 60%, progettuale 40%. Soglie: ≥ 18 orale, ≥ 16 progettuale.
- **Appello tradizionale (II appello in poi):** prova scritta individuale (5 domande + esercizi UML) + prova orale obbligatoria. Pesi: scritto $\sim 60\%$, orale $\sim 40\%$. Soglie: ≥ 18 orale, ≥ 16 scritto.
- **Laboratorio:** valutazione 0–3 punti aggiuntivi, sommati al voto finale se entrambe le prove risultano sufficienti.

1 Introduzione

1.1 Che cos'è il Software?

Software: un programma per computer e la documentazione associata (analisi dei requisiti, modelli di progetto, manuale utente, ...).

I prodotti software possono essere:

- **Generici** – sviluppati per un ampio insieme di clienti (*es. Excel, Word*);
- **Personalizzati (custom)** – sviluppati per un singolo cliente in base alle sue esigenze.

1.2 Ingegneria del Software

Ingegneria del software: disciplina che riguarda tutti gli aspetti relativi alla produzione di software, adottando un approccio sistematico e organizzato.

Campo dell'informatica che tratta sistemi software:

- complesso
- costruiti da più persone
- multiversion
- in uso per molti anni;
- che cambiano nel tempo.

Multi-person construction of multi-version software.

1.3 Il Processo di Sviluppo Software

Un insieme di attività il cui fine è lo sviluppo e l'evoluzione del software. Le attività generiche comuni a tutti i processi sono:

- **Analisi dei requisiti** – indica che cosa il sistema dovrebbe fare;
- **Progettazione** – progetto del sistema da implementare;
- **Sviluppo** – produzione del sistema software;
- **Convalida** – verifica che il software soddisfi i requisiti del cliente;
- **Evoluzione** – il software viene modificato al cambiare dei requisiti.

Il costo del software spesso supera il costo dei sistemi hardware; per questo l'ingegneria del software riguarda lo sviluppo *cost-effective* di software.

1.4 CASE: Computer-Aided Software Engineering

Sistemi software per l'automazione delle attività svolte nel processo software

- **alto livello**
supporto alle prime attività di processo: raccolta requisiti e progettazione;
- **basso livello**
supporto alle ultime attività di processo: programmazione, debugging, testing, reverse engineering.

1.5 Attributi di un Buon Software

Il software deve fornire le funzionalità e le prestazioni richieste, ed essere:

- **Mantenibile** – semplice da far evolvere e mantenere;
- **Fidato** – bassa probabilità di malfunzionamenti;
- **Efficiente** – non spreca risorse;
- **Usabile** – facile da usare;
- **Accettabile** – comprensibile, usabile e compatibile con altri sistemi.

1.6 Responsabilità Etica e Professionale

L'ingegneria del software include responsabilità più ampie della sola applicazione tecnica. Gli ingegneri del software devono comportarsi in modo onesto ed eticamente responsabile, rispettando:

- **Riservatezza** verso clienti e datori di lavoro;
- **Competenza** – non accettare lavori al di fuori delle proprie capacità;
- **Proprietà intellettuale** – rispettare copyright e brevetti;
- **Uso improprio del computer** – non usare le competenze per danneggiare terzi.

2 Analisi dei Requisiti

Esempio di tabella dei requisiti (Sistema POS):

ID	Descrizione	Tipo	Priorità
RF01	Il sistema deve elaborare vendite	F	Alta
RF02	Il sistema deve gestire pagamenti	F	Alta
RF03	Il sistema deve stampare scontrini	F	Media
RNF01	Tempo risposta < 2 secondi	NF	Alta
RNF02	Disponibilità 99.9%	NF	Media

Legenda: *F* = Funzionale, *NF* = Non Funzionale

2.1 Ingegneria dei Requisiti

Ingegneria dei requisiti: processo di ricerca, analisi, documentazione e verifica dei servizi richiesti dal cliente e dei vincoli entro i quali i servizi devono operare (spazio del problema).

Tipi di requisiti:

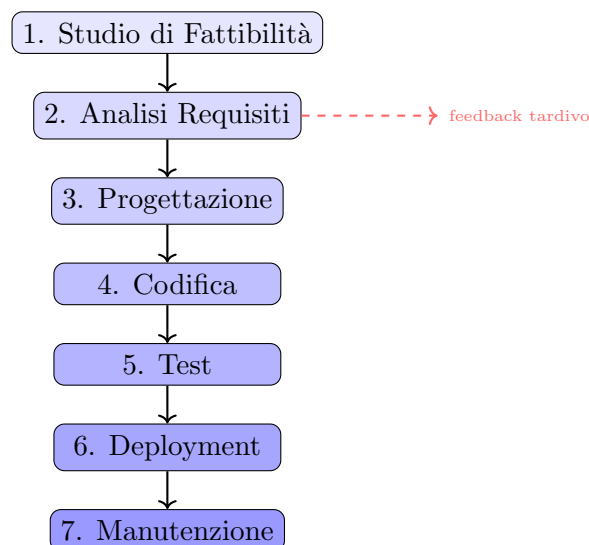
- **Funzionali** – funzionalità del sistema e comportamento atteso;
- **Non Funzionali** – qualità del sistema (prestazioni, affidabilità, sicurezza, usabilità).

2.2 Processi Software

Un **processo software** è un insieme strutturato di attività: Specificazione → Progettazione/Implementazione → Convalida → Evoluzione.

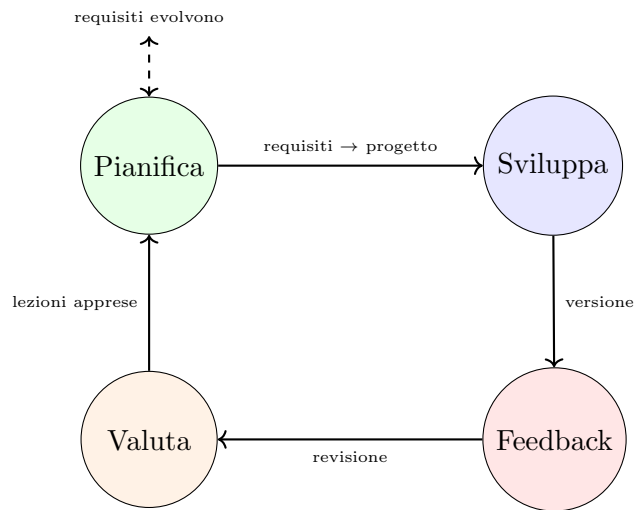
2.2.1 Modello a Cascata

Sequenza lineare: Studio di Fattibilità → Analisi Requisiti → Progettazione → Codifica → Test → Deployment → Manutenzione. **Problema:** feedback tardivo implica costi elevati.



2.2.2 Sviluppo Iterativo

Cicli frequenti di sviluppo e feedback; ogni iterazione produce una versione; i requisiti evolvono durante il progetto.



2.3 Agile

Lo **sviluppo agile** enfatizza:

- Individui e interazioni più che processi e strumenti;
- Software funzionante più che documentazione completa;
- Collaborazione con il cliente più che negoziazione di contratti;
- Rispondere ai cambiamenti più che seguire un piano.

2.3.1 Scrum

Metodo agile per gestione dello sviluppo iterativo. Tre fasi: **Pianificazione iniziale** (obiettivi, architettura) → **Cicli di sprint** (iterazioni 2-4 settimane) → **Chiusura** (documentazione, valutazione).

Ruoli:

- **Product Owner (PO)** – definisce e priorizza i requisiti nel Product Backlog; rappresenta il business; focus sul ROI;
- **Team** (3-7 sviluppatori) – auto-organizzato; seleziona requisiti dallo Sprint Backlog; implementa autonomamente;
- **Scrum Master** – garantisce che Scrum sia seguito correttamente; facilita il processo; NON è project manager (gestisce il processo, non il lavoro).

Artefatti:

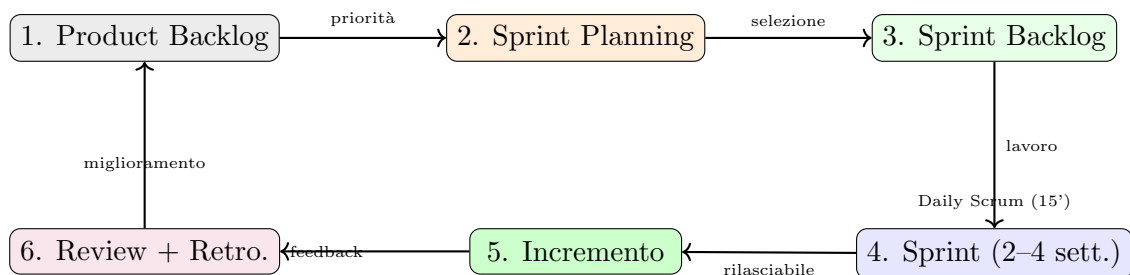
- **Product Backlog** – elenco prioritizzato di funzionalità, requisiti, user story da sviluppare;

- **Sprint Backlog** – insieme attività necessarie per realizzare i requisiti selezionati nello sprint corrente;
- **Incremento** – versione potenzialmente rilasciabile del prodotto al termine dello sprint.

Eventi/Cerimonie:

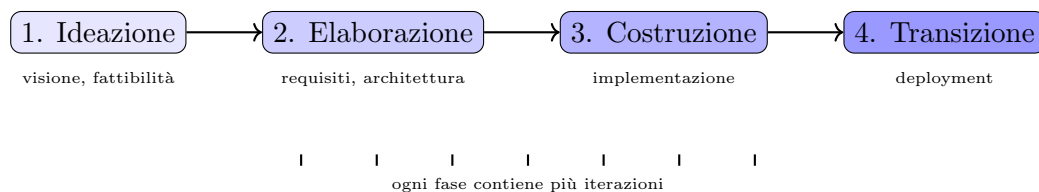
- **Sprint Planning** – pianificazione: team seleziona requisiti dal Product Backlog per lo sprint;
- **Daily Scrum** – incontro giornaliero (15 min): stato avanzamento, priorità giornaliere, impedimenti;
- **Sprint Review** – demo incremento a PO e stakeholder; raccolta feedback;
- **Sprint Retrospective** – riflessione del team su come migliorare il processo.

Velocity: stima del lavoro che il team può completare in uno sprint; base per misurare miglioramento prestazioni.



2.3.2 Unified Process (UP)

Framework iterativo in 4 fasi: **Ideazione** (visione, fattibilità, ~1 settimana), **Elaborazione** (analisi requisiti, architettura), **Costruzione** (implementazione), **Transizione** (deployment).



Ideazione: identificare la maggior parte dei casi d'uso (per nome), dettagliare il 10%, identificare rischi. Non è analisi dettagliata.

2.4 Analisi Orientata agli Oggetti (OOA)

L'**OOA** identifica i concetti/oggetti nel dominio del problema (es. *Aereo, Volo, Pilota*).

La **progettazione OO** definisce oggetti software che collaborano (es. *Aereo.getVoliEffettuati()*).

Principio: Fare la cosa giusta (analisi) e fare la cosa bene (progettazione).

3 UML: Casi d'Uso

3.1 Definizione

I **casi d'uso** sono storie scritte che descrivono come gli attori (utenti o sistemi esterni) interagiscono con il sistema.

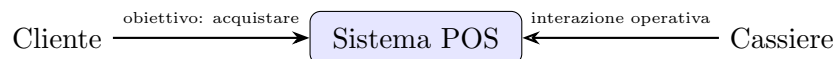
3.2 Struttura

- **Nome** – inizia con un verbo (*es. “Elabora Vendita”*);
- **Attore Primario** – chi avvia il caso d'uso;
- **Pre-condizioni** – cosa deve essere vero all'inizio;
- **Garanzia di Successo** – cosa deve essere vero se completato;
- **Scenario Principale** – scenario comune senza eccezioni;
- **Estensioni** – scenari alternativi.

3.3 Attore Primario e Obiettivo

L'**attore primario** è l'entità esterna che ha un obiettivo di business verso il sistema.

- Non coincide necessariamente con chi “tocca la UI”;
- E chi ottiene il valore principale dal completamento del caso d'uso;
- Può essere una persona, un dispositivo o un sistema esterno.



3.4 Qualità delle Estensioni

Le estensioni sono utili quando descrivono deviazioni con effetto reale sul flusso o sullo stato.

- Struttura consigliata: **condizione** → **deviazione** → **rientro/uscita**;
- Includere eccezioni tecniche o di business (*es. errore sistema, autorizzazione negata*);
- Ogni estensione deve chiarire se il caso d'uso termina, riparte o torna allo scenario principale.

3.5 I Tre Test per Validare i Casi d'Uso

3.5.1 Test del Capo

Il capo chiede: *“Cosa avete fatto tutto il giorno?”* Rispondete con il nome del caso d'uso. È soddisfatto?

Validi: “Elabora Vendita”, “Gestisci Reso”. *Non validi:* “Inserisci Articolo” (troppo piccolo).

3.5.2 Test EBP (Elementary Business Process)

EBP: attività che aggiunge **valore di business misurabile** e lascia i dati in **stato coerente**.

Valido: “Elabora Vendita” (vendita registrata, articoli aggiornati). *Non valido:* “Visualizza Prodotto” (nessun valore/stato coerente).

3.5.3 Test della Dimensione

Più di 1 azione; da 3 a 10 pagine nel formato dettagliato. Attività secondarie possono diventare casi d’uso separati (i tre test possono essere violati).

3.6 Diagramma dei Casi d'Uso

Il **diagramma dei casi d'uso** è un modello UML che mostra gli attori e le funzionalità del sistema. I costrutti del diagramma sono:

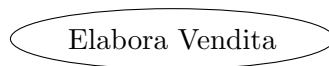
3.6.1 Attore

Persona o sistema esterno; rappresentato con figura stilizzata.



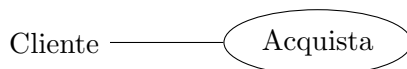
3.6.2 Caso d'Uso

Funzionalità offerta dal sistema; nome con verbo all'infinito.



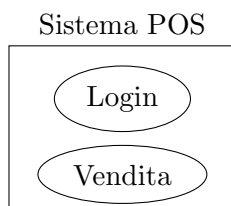
3.6.3 Associazione

Linea continua che collega attore e caso d'uso; indica interazione.



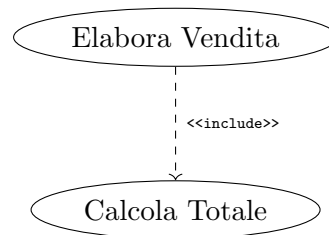
3.6.4 System Boundary

Rettangolo che delimita il confine del sistema.



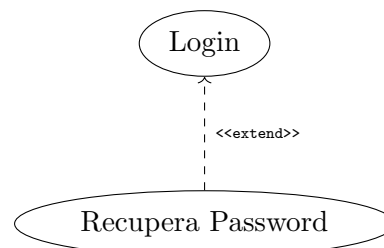
3.6.5 Include

Relazione tra casi d'uso: il caso base include sempre il caso incluso (freccia tratteggiata con <<include>>).



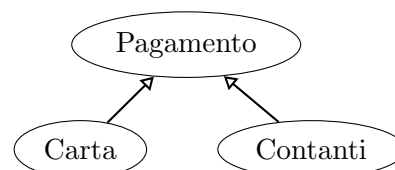
3.6.6 Extend

Relazione tra casi d'uso: il caso estensione estende opzionalmente il caso base (freccia tratteggiata con <<extend>>).



3.6.7 Generalization

Relazione di generalizzazione tra casi d'uso: freccia continua con punta vuota verso il caso generico.



Nota: I casi d'uso sono *documenti di testo*; il diagramma è solo una vista d'insieme.

3.7 Modello di Dominio

Rappresentazione visuale di classi concettuali/oggetti del mondo reale (spazio del problema, non soluzione software).

Elementi: **Classi Concettuali** (es. *Sale*, *Payment*), **Attributi** (es. *Sale.date*), **Associazioni** (es. *Sale "pays-for" Payment*), **Generalizzazione** (es. *CreditCard* è *Payment*). Il modello ispira le classi di progetto e riduce il **salto rappresentazionale** tra come gli stakeholder concepiscono il dominio e come è rappresentato nel software.

3.8 Diagrammi di Sequenza di Sistema (SSD)

Un **diagramma di sequenza di sistema (SSD)** mostra, per un particolare scenario di un caso d'uso, gli eventi generati dagli attori esterni al sistema, il loro ordine e gli eventi inter-sistema. È un artefatto di **progettazione ad alto livello**: descrive **cosa** il sistema deve fare visto dall'esterno, non **come** è realizzato internamente.

Elementi:

- **Attori esterni** – interagiscono direttamente con il sistema;
- **Sistema** – rappresentato come scatola nera;
- **Eventi di sistema** – input generati dagli attori;
- **Valori di ritorno** – output del sistema (opzionali).

Eventi e operazioni di sistema:

- Un **evento di sistema** è un evento esterno, di input, generato da un attore;
- Un'**operazione di sistema** è l'operazione che il sistema esegue in risposta all'evento;
- I nomi devono essere astratti (intenzione, non azione) e iniziare con un verbo (*es. enterItem, makePayment*).

Linea guida: disegnare un SSD per lo scenario principale di successo di ciascun caso d'uso e per gli scenari alternativi più frequenti o complessi.

3.9 Contratti delle Operazioni

I **contratti** descrivono il comportamento del sistema in modo dettagliato, usando pre-condizioni e post-condizioni per specificare i cambiamenti agli oggetti del modello di dominio.

Sezioni di un contratto:

- **Operazione** – nome e parametri;
- **Riferimenti** – casi d'uso correlati;
- **Pre-condizioni** – stato del sistema prima dell'esecuzione;
- **Post-condizioni** – cambiamenti di stato dopo l'esecuzione.

Post-condizioni (scritte al passato):

- Creazione o cancellazione di istanze;
- Formazione o rottura di collegamenti (associazioni);
- Modifica di attributi.

Metafora del palcoscenico: si confrontano due “foto” dello stato del sistema prima e dopo l'operazione; le post-condizioni descrivono i cambiamenti osservati.

3.10 Tracciabilità: Casi d'Uso → SSD → Contratti

Il percorso continuo dal requisito alla specifica operativa.



Chi fa cosa? → Cosa cambia?

- Ogni **caso d'uso** produce uno o più scenari;
- Ogni scenario diventa un **SSD** (scatola nera);
- Ogni operazione di sistema sull'SSD genera un **contratto**;
- Il contratto alimenta il **progetto**: le post-condizioni dicono cosa creare/collegare/modificare.

3.11 Template pratico di contratto

Contratto
Operazione: nome(parametri) : tipo Riferimenti: UC..., SSD... Pre-condizioni: • lo stato iniziale richiesto Post-condizioni: • new / delete / link / set • oggetti creati / modificati / eliminati

4 Diagrammi di Dominio

I **diagrammi di dominio** rappresentano classi concettuali del mondo reale (spazio del problema). A differenza dei diagrammi di progetto (DCD), mostrano solo classi con attributi, **senza metodi**.

4.1 Struttura di una Classe di Dominio

Nel modello di dominio una classe ha solo: **Nome** e **Attributi** (stato). I metodi appartengono al modello di progetto.

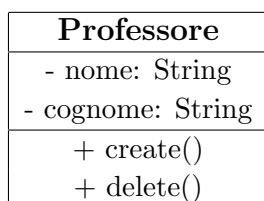
Visibilità: + public, # protected, - private.

“One class should do one thing and do it well.”

4.2 Notazione dei Diagrammi di Classe

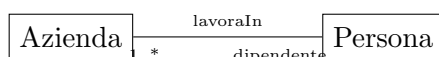
4.2.1 Classe

Rettangolo diviso in tre sezioni: nome, attributi, metodi.



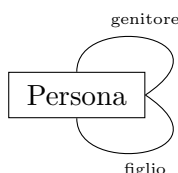
4.2.2 Associazione

Linea che collega due classi; decorata con nome, ruoli e molteplicità.



4.2.3 Associazione Riflessiva

Associazione sulla stessa classe; ruoli obbligatori.



4.2.4 Aggregazione

Relazione “parte-di” debole; rombo vuoto.



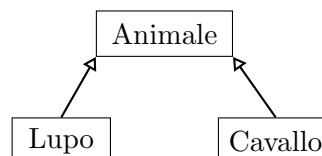
4.2.5 Composizione

Relazione “parte-di” forte; rombo pieno. Le parti vivono e muoiono con l'intero.



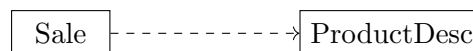
4.2.6 Generalizzazione

Relazione “è-un-tipo-di”; freccia con punta vuota verso la superclasse.



4.2.7 Dipendenza

Freccia tratteggiata; indica che il cliente usa il fornitore.



4.2.8 Interfaccia

Notazione lollipop (fornita) o socket (richiesta).



4.3 Vincoli sulle Generalizzazioni

- **disjoint** (default) / **overlapping** – sottoclassi senza/con istanze comuni;
- **incomplete** (default) / **complete** – superclasse con/senza istanze proprie.

Ereditarietà multipla: possibile solo se esiste una sottoclasse comune tra due sottoclassi non disgiunte.

4.4 Classi Astratte e Polimorfismo

Una **classe astratta** (nome in *corsivo*) contiene metodi senza implementazione e non può essere istanziata. Il **polimorfismo** permette implementazioni diverse dello stesso metodo nelle sottoclassi.

4.5 Aggregazione vs. Ereditarietà

L'ereditarietà è la forma più forte di interdipendenza. **Preferire l'aggregazione** quando:

- Il comportamento può cambiare a runtime (*es. promozione di un impiegato*);
- Un oggetto può avere più ruoli contemporaneamente.

5 Diagrammi di Sequenza

I **diagrammi di sequenza** mostrano le interazioni tra oggetti nel tempo, evidenziando l'ordine temporale dei messaggi scambiati.

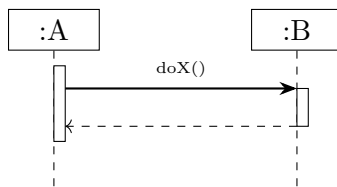
5.1 Elementi Principali

- **Partecipanti** – oggetti o attori rappresentati da rettangoli in alto;
- **Linea di vita (lifeline)** – linea verticale tratteggiata;
- **Barra di attivazione** – rettangolo sulla lifeline che indica esecuzione attiva;
- **Messaggi** – frecce orizzontali tra lifeline.

5.2 Notazione dei Diagrammi di Sequenza

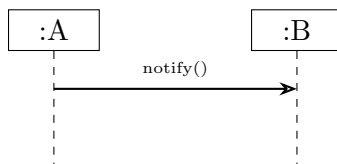
5.2.1 Messaggio Sincrono

Freccia piena; il mittente attende risposta.



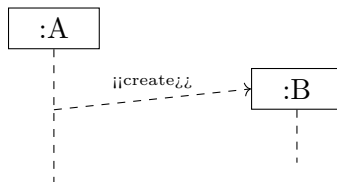
5.2.2 Messaggio Asincrono

Freccia aperta; il mittente continua.



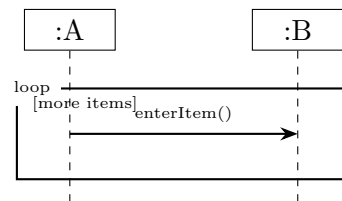
5.2.3 Creazione Oggetto

Freccia verso il rettangolo del nuovo oggetto.



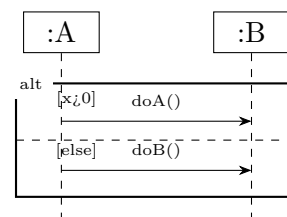
5.2.4 Frammento loop

Iterazione finché la guardia è vera.



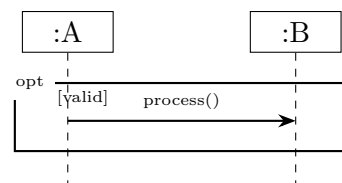
5.2.5 Frammento alt

Alternativa con guardie.



5.2.6 Frammento opt

Esecuzione opzionale.



5.3 Da SSD a Diagrammi di Progetto

- Gli **SSD** mostrano il sistema come scatola nera: fissano il comportamento atteso dall'esterno nelle fasi iniziali di progettazione;

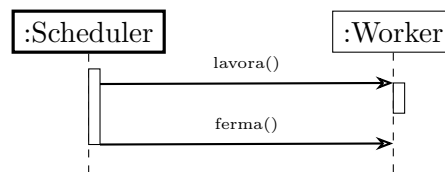
- I **diagrammi di sequenza di progetto** mostrano le interazioni interne tra oggetti software e quindi la soluzione progettuale concreta;
- Le operazioni di sistema degli SSD diventano metodi delle classi controller.

Differenza chiave: SSD = **ricetta** (intenzione e passi attesi); sequenza di progetto = **piatto finito** (oggetti reali, messaggi reali, responsabilità assegnate).

Relazione con diagrammi di classe: i messaggi corrispondono a operazioni nelle classi; i due diagrammi devono essere consistenti.

5.4 Oggetti attivi e sincrono/asincrono

- **Oggetto attivo:** possiede un proprio thread di controllo (lifeline con bordo spesso);
- **Messaggio sincrono:** mittente bloccato finché il destinatario non risponde (freccia piena);
- **Messaggio asincrono:** mittente continua senza attendere (freccia aperta);
- Usare asincrono per eventi, notifiche, code di messaggi; sincrono per operazioni che richiedono un risultato immediato.



6 Architettura Logica

L'**architettura logica** è l'organizzazione su larga scala delle classi software in package, sottosistemi e strati. È "logica" perché non si prendono decisioni sulla disposizione fisica dei componenti (architettura di deployment).

6.1 Strati (Layers)

Uno **strato** è un gruppo a grana grossa di classi, package o sottosistemi con responsabilità coese rispetto a un aspetto importante del sistema.

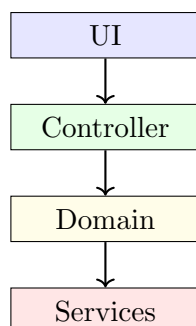
Caratteristiche:

- Gli strati più alti utilizzano i servizi degli strati più bassi (ma non viceversa);
- Strati inferiori = servizi generali e di basso livello;
- Strati superiori = più specifici per l'applicazione.

6.2 Architettura Stretta vs. Rilassata

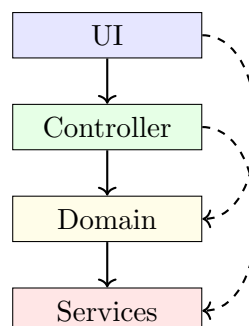
6.2.1 Architettura Stretta

Uno strato può richiamare **solo** i servizi dello strato immediatamente più basso.



6.2.2 Architettura Rilassata

Uno strato può richiamare i servizi di **qualsiasi** strato più in basso.

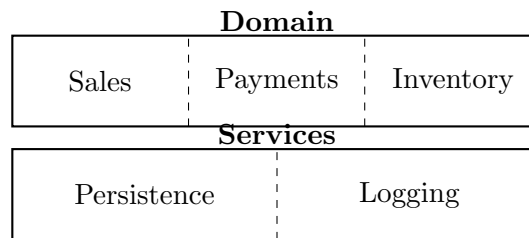


6.3 Architettura Tipica a Quattro Strati

Strato	Responsabilità
UI (Presentation)	Gestisce le interazioni con l'utente: finestre GUI, pagine web, report. Delega le operazioni al Controller.
Controller (Application)	Coordina l'esecuzione delle operazioni applicative, gestisce il workflow e lo stato di sessione. Interagisce con Domain.
Domain (Business Logic)	Contiene la logica di business e gli oggetti di dominio. Implementa le regole applicative.
Services (Technical)	Offre servizi tecnici: persistenza, logging, sicurezza, comunicazione di rete.

6.4 Partizioni

Una **partizione** è una divisione orizzontale di un sottosistema all'interno di uno strato. Permette di organizzare funzionalità correlate ma distinte.



6.5 Diagrammi dei Package UML

L'architettura logica si rappresenta con **diagrammi dei package**:

- Uno strato è modellato come un package UML;
- Le **dipendenze** tra package si mostrano con frecce tratteggiate;
- I package possono essere annidati (namespace): `Domain::Sales`.

6.6 Benefici dell'Architettura a Strati

- **Separazione degli interessi** – dipendenze ridotte, coesione migliorata;
- **Riusabilità** – strati inferiori riusabili in altri progetti;
- **Sostituibilità** – uno strato può essere rimpiazzato senza impattare gli altri;
- **Sviluppo in team** – segmentazione logica facilita il lavoro parallelo;
- **Testabilità** – ogni strato può essere testato indipendentemente.

6.7 Principio di Separazione Modello-Vista

Gli oggetti dello strato Domain (**modello**) non devono essere accoppiati direttamente agli oggetti UI (**vista**):

- Non mettere logica applicativa nei metodi degli oggetti UI;
- Gli oggetti UI devono solo: inizializzare l'interfaccia, ricevere eventi, delegare le richieste al Domain.

Nota: il pattern *Observer* permette un rilassamento controllato di questo principio.

7 Verso la progettazione a oggetti

La progettazione OO in pratica combina **modellazione leggera** e **codifica iterativa**: si modella per capire/comunicare, poi si valida rapidamente nel codice.

7.1 Tre approcci pratici

- **Codifica diretta** – progettare mentre si implementa (*es. con TDD e refactoring*);
- **Disegno + codifica** – brevi sessioni UML (lavagna o CASE) prima di passare all'IDE;
- **Solo disegno** – approccio generalmente poco usato nei contesti agili.

7.2 Agile Modeling

- Modellare per **comprendere e comunicare**, non per documentare in eccesso;
- Modellare in gruppo;
- Creare modelli **statici** e **dinamici** in parallelo;
- Limitare il tempo di disegno all'inizio dell'iterazione, poi iterare codice–modello.

7.3 Modelli statici e dinamici

- **Dinamici**: diagrammi di sequenza/comunicazione, utili per responsabilità, collaborazione tra oggetti e logica dei metodi;
- **Statici**: diagrammi delle classi/package, utili per struttura, dipendenze e firme delle operazioni.

Idea chiave: la notazione UML è un mezzo; il valore è nelle decisioni di progettazione (responsabilità, principi GRASP, pattern).

8 Diagrammi di Interazione

I diagrammi di interazione rappresentano **come** gli oggetti collaborano scambiando messaggi per realizzare un comportamento.

8.1 Sequenza vs. Comunicazione

- **Diagramma di sequenza**: enfatizza l'ordine temporale dei messaggi;
- **Diagramma di comunicazione**: enfatizza i collegamenti tra oggetti e numera i messaggi per mostrare l'ordine logico.

8.2 Messaggi e responsabilità

- Un messaggio invoca un'operazione del destinatario;
- Le decisioni su “chi fa cosa” guidano l'assegnazione delle responsabilità;
- I messaggi condizionali usano **guardie** (es. $[totale > 0]$);
- Le iterazioni sono espresse con frammenti **loop** o numerazioni ripetute.

8.3 Frammenti combinati più usati

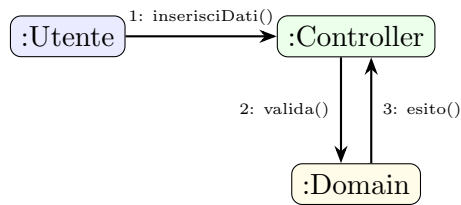
- **alt** – alternativa mutuamente esclusiva;
- **opt** – comportamento opzionale;
- **loop** – ripetizione finché la guardia è vera;
- **par** – rami eseguiti in parallelo;
- **neg** – scenario non valido.

8.4 Relazione con i diagrammi di classe

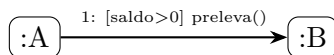
Ogni messaggio dovrebbe corrispondere a un'operazione nelle classi di progetto: i diagrammi di interazione aiutano quindi a **derivare e verificare** firme/metodi nel diagramma delle classi.

8.5 Notazione compatta (come disegnarli)

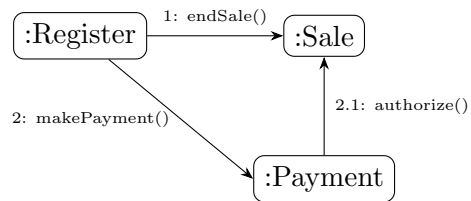
8.5.1 Comunicazione



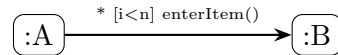
8.5.2 Messaggio condizionale



8.5.3 Numerazione annidata



8.5.4 Messaggio iterativo



8.6 Checklist di coerenza

Prima di passare al codice, verificare:

- Ogni messaggio ha un **mittente** e un **destinatario** espliciti;
- Ogni messaggio corrisponde a un'operazione nella **classe destinataria** del diagramma di classe;
- La numerazione nei diagrammi di comunicazione è **progressiva e annidata** correttamente;
- Le **guardie** sono mutuamente esclusive nei frammenti **alt**;
- I **loop** hanno una condizione di terminazione raggiungibile;
- Attori e oggetti sono **coerenti** con SSD e modello di dominio.

9 Diagrammi di Classe di Progetto

Il diagramma di classe di progetto descrive la vista statica del software: classi, attributi, operazioni e relazioni implementative.

9.1 Contenuto della classe UML

- **Nome classe**;
- **Attributi** (stato);
- **Operazioni** (comportamento).

Notazione operazioni: `visibilita nome(parametri): tipo_ritorno [property]`
con visibilità + pubblica, # protetta, - privata.

9.2 Dal modello di dominio al design

- In analisi si identificano concetti e relazioni del dominio;
- In progetto si raffinano associazioni, molteplicità e navigabilità in funzione del software;
- Il principio guida resta: *una classe dovrebbe fare una cosa e farla bene*.

9.3 Relazioni e raffinamenti tipici

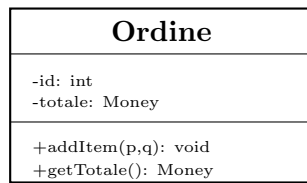
- Associazioni bidirezionali o unidirezionali (se una sola classe deve conoscere l'altra);
- Classi associative quando la relazione ha attributi propri;
- Qualificatori per indicizzare l'accesso a insiemi associati;
- Generalizzazioni con vincoli **disjoint/overlapping** e **complete/incomplete**.

9.4 Classi e operazioni astratte

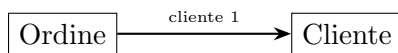
Una classe con almeno un'operazione astratta è **astratta** e non istanziabile; le sottoclassi concrete devono fornire implementazioni coerenti (polimorfismo).

9.5 Notazione compatta (come disegnarli)

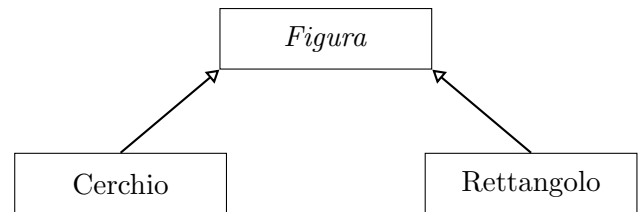
9.5.1 Classe UML



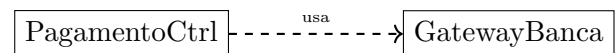
9.5.2 Associazione unidirezionale



9.5.3 Classe astratta



9.5.4 Dipendenza



10 Diagrammi di Attività

I diagrammi di attività modellano flussi di lavoro complessi, quando un caso d'uso testuale non rende bene sequenze, parallelismi e decisioni.

10.1 Elementi principali

- **Action** – passo operativo;
- **Control flow** – arco che collega le azioni;
- **Object node / object flow** – dati prodotti/consumati;
- **Initial node** e **final node**;
- **Swimlane (partitions)** – responsabilità per attore/ruolo/sottosistema.

10.2 Controllo del flusso

- **Decision** – diramazione con guardie;
- **Merge** – ricongiunge alternative;
- **Fork** – avvia rami paralleli;
- **Join** – sincronizza rami paralleli.

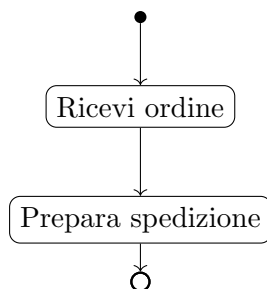
Nota: merge e join non sono equivalenti: il merge continua con un solo input, il join richiede tutti gli input.

10.3 Uso consigliato

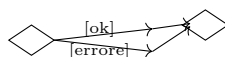
- Processi con più partecipanti e passaggi concorrenti;
- Workflow con scambi di dati espliciti;
- Decomposizione gerarchica di attività complesse in sotto-attività.

10.4 Notazione compatta (come disegnarli)

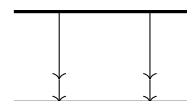
10.4.1 Flusso base



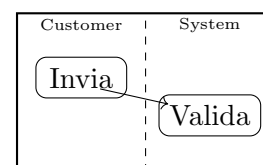
10.4.2 Decision + Merge



10.4.3 Fork + Join

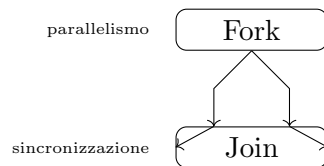


10.4.4 Swimlane



10.5 Regole operative importanti

- **Decision + Merge:** la decisione apre rami con guardie; il merge li richiude. Non confondere merge (**un** input atteso) con join (**tutti** gli input attesi).
- **Fork + Join:** fork avvia parallelismo, join sincronizza. Il join attende **tutti** i rami in ingresso.
- **Object flow:** un'azione produce un dato (object node) che un'azione successiva consuma. Utile per modellare dipendenze dati esplicite.
- **Pin:** oggetto attaccato direttamente a un'azione (input/output pin) invece di nodo separato.
- **Swimlane:** ogni attore/ruolo ha la propria colonna; le azioni vanno nella colonna di chi le esegue.



11 Diagrammi di Stato (Statechart)

I diagrammi di stato descrivono il comportamento di oggetti **dipendenti dallo stato**, mostrando come reagiscono agli eventi e come evolvono nel tempo.

11.1 Concetti base

- **Stato**: configurazione significativa dei valori dell'oggetto;
- **Transizione**: passaggio tra stati attivato da un evento;
- **Evento**: accadimento che può innescare una transizione;
- **Azione**: computazione breve eseguita durante la transizione;
- **Guardia**: condizione booleana che abilita la transizione.

Forma compatta: evento(parametri) [guardia] / azione.

11.2 Quando usarli

- Oggetti reattivi complessi (*es. dispositivi, ordini, pratiche*);
- Modellazione di protocolli e sequenze valide di operazioni;
- Controllo del flusso UI/sessione lato server.

11.3 Stati composti

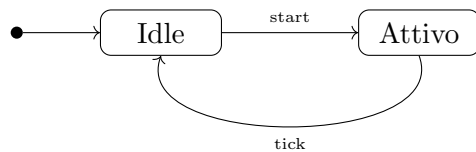
Gli statechart UML supportano stati composti (sotto-stati annidati) e regioni ortogonali per modellare comportamenti concorrenti; questo estende la potenza degli automi a stati finiti classici.

11.4 Linee guida pratiche

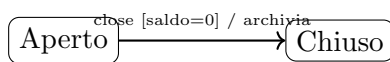
- Modellare solo cambiamenti di stato **significativi** per il dominio;
- Evitare dettagli irrilevanti che non modificano il comportamento osservabile;
- Usare i diagrammi di stato insieme a casi d'uso e diagrammi di interazione per mantenere coerenza tra vista statica e dinamica.

11.5 Notazione compatta (come disegnarli)

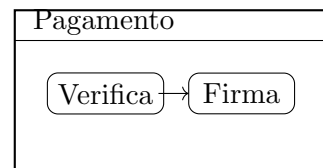
11.5.1 Stati e transizioni



11.5.2 Transizione con guardia



11.5.3 Stato composto

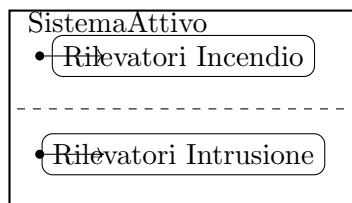


11.5.4 Stato finale

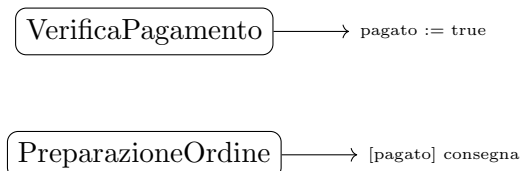


11.6 Schemi visuali avanzati

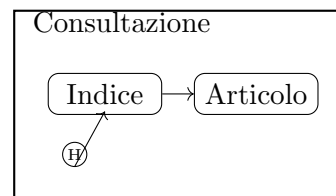
11.6.1 Stato ortogonale



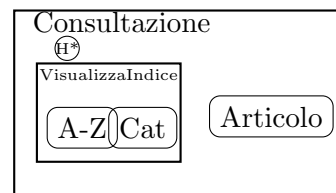
11.6.2 Comunicazione asincrona



11.6.3 History semplice H



11.6.4 History multilivello H*



12 Principi GRASP

I principi GRASP (General Responsibility Assignment Software Patterns) sono linee guida fondamentali per l'assegnazione delle responsabilità nella progettazione orientata agli oggetti.

12.1 Creator

Problema: Chi dovrebbe essere responsabile della creazione di una nuova istanza di una classe? Soluzione: assegnare alla classe B la responsabilità di creare un'istanza di A se una o più delle seguenti condizioni è vera:

- B aggrega gli oggetti A.
- B contiene gli oggetti A.
- B registra le istanze degli oggetti A.

- B ha i dati di inizializzazione che saranno passati ad A al momento della creazione (quindi B è un Esperto rispetto alla creazione di A).

Esempio: poiché Sale contiene molti oggetti SalesLineItem, il pattern Creator suggerisce che Sale è un buon candidato per la responsabilità di creare SalesLineItem.

12.2 Information Expert (Esperto)

Problema: Qual è il principio generale dell'attribuzione delle responsabilità agli oggetti?

Soluzione: assegnare la responsabilità alla classe che ha le informazioni necessarie per soddisfarla. Discussione: l'esperto chiede di trovare l'oggetto che possiede la maggior parte delle informazioni richieste per la responsabilità. Se ignoriamo i consigli dell'esperto, l'accoppiamento complessivo aumenta.

12.3 Low Coupling (Accoppiamento Basso)

Problema: Dove assegnare la responsabilità per evitare un accoppiamento eccessivo?

Soluzione: assegnare la responsabilità in modo che l'accoppiamento rimanga basso. Valutazione: l'accoppiamento è una misura di quanto fortemente un elemento è connesso ad altri elementi. Un basso accoppiamento aumenta la riusabilità e facilita i cambiamenti.

12.4 High Cohesion (Alta Coesione)

Problema: Come mantenere gli oggetti focalizzati, comprensibili e gestibili? Soluzione: assegnare una responsabilità in modo che la coesione rimanga alta. Discussione: la coesione è una misura di quanto fortemente siano correlate e concentrate le responsabilità di un elemento. Una classe con responsabilità altamente correlate che esegue una quantità adeguata di lavoro ha una coesione alta.

12.5 Controller

Problema: Chi dovrebbe essere responsabile della gestione di un evento di sistema?

Soluzione: assegnare la responsabilità a un oggetto che rappresenta il sistema complessivo, un oggetto radice, un sottosistema principale o uno scenario di caso d'uso. Discussione: il controller è il primo oggetto oltre lo strato di UI che riceve e coordina l'esecuzione di un'operazione di sistema. Deve delegare il lavoro ad altri oggetti piuttosto che svolgerlo direttamente.

12.6 Polimorfismo

Problema: Come gestire le variazioni di comportamento basate sul tipo? Soluzione: assegnare la responsabilità per il comportamento variabile ai tipi per cui il comportamento varia, utilizzando operazioni polimorfiche invece di ramificazioni esplicite basate sul tipo.

12.7 Protected Variazioni

Problema: Come progettare oggetti, sottosistemi e sistemi in modo che le variazioni o l'instabilità in questi elementi non abbiano un impatto indesiderato su altri elementi?

Soluzione: identificare i punti di variazione o instabilità previsti e assegnare le responsabilità per creare un'interfaccia stabile attorno a essi, utilizzando il polimorfismo per creare varie implementazioni di questa interfaccia.

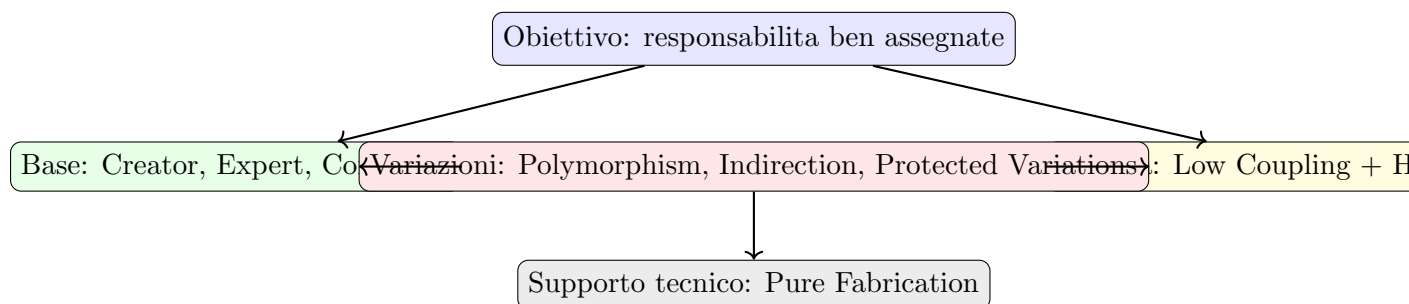
12.8 Pure Fabrication (Fabbricazione Pura)

Problema: Cosa fare quando la soluzione suggerita dall'esperto porta a un basso accoppiamento ma anche a una bassa coesione? Soluzione: creare una classe artificiale che non rappresenta un concetto del problema dominio, specificamente progettata per raggiungere basso accoppiamento, alta coesione e riusabilità. Esempio: un controller di caso d'uso è una specie di Pure Fabrication.

12.9 Indirizione

Problema: Dove assegnare la responsabilità per evitare un accoppiamento diretto tra due o più elementi? Soluzione: assegnare la responsabilità a un oggetto intermedio per mediare tra altri componenti o servizi in modo che non siano direttamente accoppiati. Discussione: l'intermediario crea un'indirizione tra gli altri componenti, supportando basso accoppiamento e potenziale di riuso.

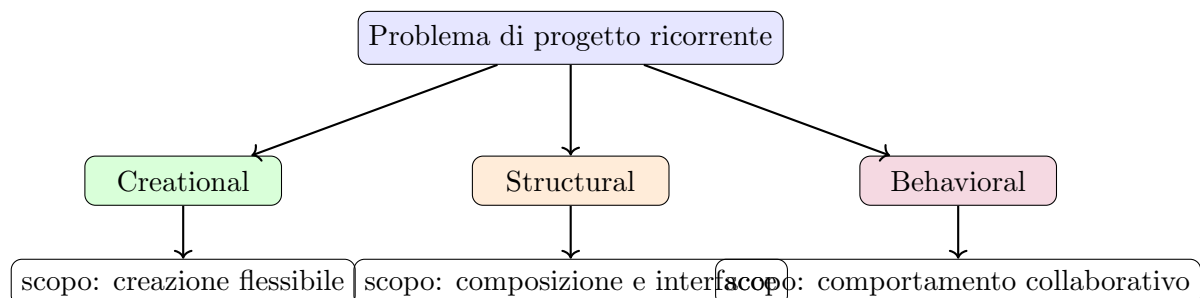
12.10 Mappa visuale dei principi GRASP



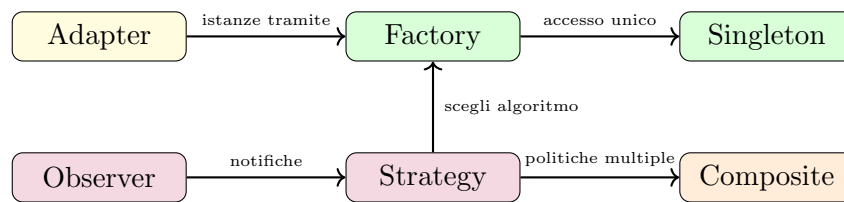
13 Design Patterns

I design pattern sono soluzioni progettuali comuni a problemi di progettazione ricorrenti. I 23 design pattern GoF sono classificati in base al loro scopo (creazionale, strutturale o comportamentale).

13.1 Mappa visuale: scopo dei pattern



13.2 Combinazioni frequenti (visione pratica)



13.3 Creational Patterns

Questi pattern riguardano il processo di istanziazione degli oggetti, rendendo il sistema indipendente da come gli oggetti vengono creati, rappresentati e compost.

Factory Method Definisce un'interfaccia per creare un oggetto ma lascia decidere alle sottoclassi quale classe istanziare.

Abstract Factory Fornisce un'interfaccia per creare famiglie di oggetti correlati senza specificare le loro classi concrete.

Builder Separa la costruzione di un oggetto complesso dalla sua rappresentazione per poter utilizzare lo stesso processo di costruzione per diverse rappresentazioni.

Prototype Specifica il tipo di oggetto da creare usando un'istanza prototipica e crea nuovi oggetti copiando questo prototipo.

Singleton Assicura che una classe abbia una sola istanza e fornisce un punto di accesso globale ad essa.

13.4 Structural Patterns

Questi pattern riguardano la composizione di classi e oggetti per formare strutture più grandi.

Adapter Converte l'interfaccia di una classe in un'altra richiesta dal client, permettendo a classi con interfacce incompatibili di lavorare insieme.

Bridge Disaccoppia un'astrazione dalla sua implementazione permettendo loro di variare indipendentemente.

Composite Compone oggetti in strutture ad albero per rappresentare gerarchie tutto-parti.

Decorator Aggiunge dinamicamente responsabilità aggiuntive a un oggetto, fornendo un'alternativa flessibile all'utilizzo di sottoclassi per estendere funzionalità.

Facade Fornisce un'interfaccia unificata a un insieme di interfacce in un sottosistema, rendendo più facile il loro utilizzo.

Flyweight Usa la condivisione per supportare efficacemente un gran numero di oggetti di granularità fine.

Proxy Fornisce un rappresentante o un segnaposto per un altro oggetto per controllarne l'accesso.

13.5 Behavioral Patterns

Questi pattern riguardano l'assegnazione di responsabilità tra gli oggetti e la comunicazione tra essi.

Chain of Responsibility Evita l'accoppiamento tra il mittente e il ricevitore di una richiesta permettendo a più oggetti di provare a gestirla.

Command Incapsula una richiesta come oggetto, permettendo la parametrizzazione dei client con code di richieste, la registrazione e l'annullamento delle operazioni.

Interpreter Dato un linguaggio, definisce una rappresentazione grammaticale e un interprete per tale linguaggio.

Iterator Fornisce un modo per accedere sequenzialmente agli elementi di un oggetto aggregato senza esporre la rappresentazione sottostante.

Mediator Definisce un oggetto che incapsula come un insieme di oggetti interagiscono, promuovendo il basso accoppiamento impedendo agli oggetti di riferirsi esplicitamente l'un l'altro.

Memento Senza violare l'incapsulamento, cattura e esternalizza lo stato interno di un oggetto in modo che possa essere ripristinato successivamente in tale stato.

Observer Definisce una dipendenza uno-a-molti tra oggetti in modo che quando un oggetto cambia stato, tutti i suoi dipendenti vengano notificati e aggiornati automaticamente.

State Consente a un oggetto di modificare il proprio comportamento quando il suo stato interno cambia, facendo apparire l'oggetto come se cambiasse classe.

Strategy Definisce una famiglia di algoritmi, li incapsula ciascuno e li rende intercambiabili, permettendo di variare l'algoritmo utilizzato indipendentemente dal contesto di utilizzo.

Template Method Definisce lo scheletro di un algoritmo in un'operazione, posticipando alcuni passaggi alle sottoclassi.

Visitor Rappresenta un'operazione da eseguire sugli elementi di una struttura di oggetti, permettendo di definire una nuova operazione senza modificare le classi degli elementi su cui opera.

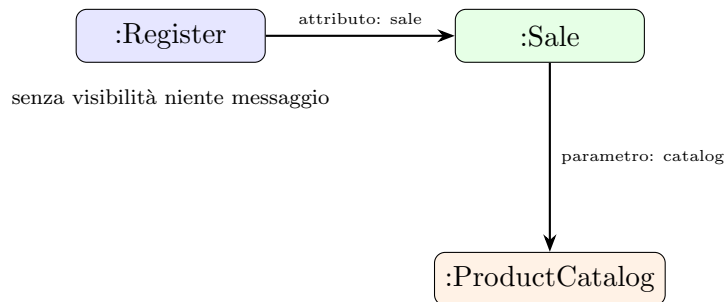
14 Dalla progettazione al codice

14.1 Progettare la visibilità

Stabilire quali relazioni di navigazione esistono tra oggetti per permettere a un oggetto mittente di inviare messaggi a un oggetto destinatario.

Forme di visibilità:

- **Attributo (attribute visibility):** B è un attributo di A – la più frequente e stabile: `class A { private B b; }`
- **Parametro (parameter visibility):** B è passato come parametro a un metodo di A – temporanea, locale;
- **Locale (local visibility):** B è dichiarato come variabile locale in un metodo di A;
- **Globale (global visibility):** B è accessibile globalmente ad A (es. singleton, registry) – da usare con cautela.



Linea guida: preferire visibilità per attributo; usare parametro/locale per collaborazioni temporanee; evitare visibilità globale.

14.2 Assegnazione delle responsabilità e tipi di visibilità

La scelta della visibilità dipende dal tipo di responsabilità:

- **Creazione (Creator)** → visibilità per attributo o locale;
- **Registrazione/notifica** → visibilità per parametro (es. Observer);
- **Interrogazione stato** → visibilità per attributo (Information Expert);
- **Coordinamento flusso** → visibilità per attributo su controller, parametro su oggetti transienti.

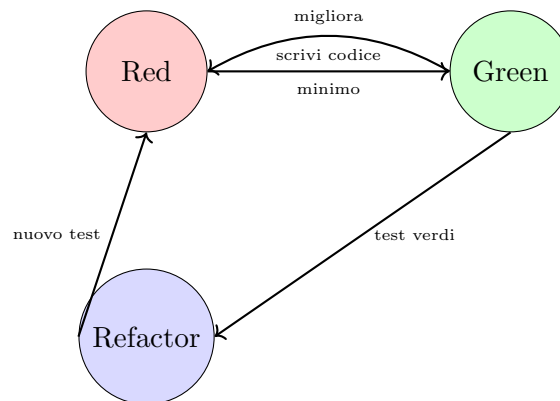
Importante: la visibilità non è solo “chi conosce chi”, ma **quanto a lungo** serve la conoscenza.

14.3 TDD: Test-Driven Development

Il Test-Driven Development è una tecnica di sviluppo in cui i test vengono scritti **prima** del codice di produzione.

Ciclo TDD (Red-Green-Refactor):

- **Red:** scrivere un test che fallisce (definisce il requisito);
- **Green:** scrivere il minimo codice per far passare il test;
- **Refactor:** migliorare il codice mantenendo i test verdi.



Vantaggi:

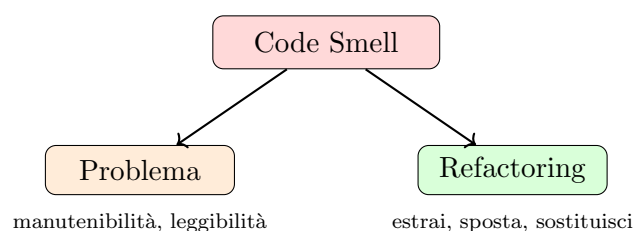
- Riduce i difetti (test come rete di sicurezza);
- Guida il progetto: il test è lo “scenario d’uso” dell’API;
- Incoraggia codice modulare e disaccoppiato (se è difficile testare, è mal progettato).

Linee guida:

- Test **automatici** e **ripetibili**;
- Un test per **un** comportamento (non testare più cose insieme);
- Nomi descrittivi: `testMetodo_condizione_risultato`;
- Usare **mock/stub** per isolare il componente sotto test.

14.4 Refactoring e Code Smells

Il **refactoring** è la modifica del codice per migliorarne la struttura interna senza alterarne il comportamento esterno.



14.4.1 Code Smells principali

- **Duplicate Code** – stesso codice in più punti → **Extract Method**;
- **Long Method** – metodo troppo lungo → **Extract Method**, **Replace Temp with Query**;
- **Large Class** – classe con troppe responsabilità → **Extract Class**;
- **Feature Envy** – un metodo usa più dati di un'altra classe → **Move Method**;
- **Switch Statement** – logica basata su tipo → **Replace Conditional with Polymorphism**;
- **Primitive Obsession** – uso eccessivo di tipi primitivi → **Replace Data Value with Object**;
- **Shotgun Surgery** – una modifica richiede cambiamenti in molte classi → **Move Method/Field**;
- **Inappropriate Intimacy** – classi troppo accoppiate → **Change Bidirectional to Unidirectional**;
- **Lazy Class** – classe che fa troppo poco → **Collapse Hierarchy**, **Inline Class**;
- **Speculative Generality** – codice per casi futuri mai usati → **Collapse Hierarchy**, **Remove Parameter**;
- **Temporary Field** – attributi usati solo in certi casi → **Extract Class**;
- **Message Chain** – chiamate incatenate (`a.getB().getC().getD()`) → **Hide Delegate**;
- **Middle Man** – classe che si limita a delegare → **Remove Middle Man**;
- **Divergent Change** – una classe cambia per ragioni diverse → **Extract Class**;
- **Parallel Inheritance Hierarchies** – creare una classe richiede crearne un'altra → **Collapse Hierarchy**;
- **Data Class** – classe solo con attributi (getter/setter) → **Move Method**, **Encapsulate Field**;
- **Refused Bequest** – sottoclasse che non usa metodi ereditati → **Replace Inheritance with Delegation**.

14.4.2 Tecniche di refactoring fondamentali

- **Extract Method** – trasformare un blocco in metodo separato;
- **Extract Class** – dividere una classe in due classi coese;
- **Move Method / Move Field** – spostare dove appartengono logicamente;
- **Replace Conditional with Polymorphism** – sostituire if/else su tipo con sottoclassi;
- **Rename Method / Rename Variable** – migliorare i nomi;

- **Encapsulate Field** – rendere privato con getter/setter;
- **Pull Up / Push Down** – spostare nella superclasse o sottoclasse.

Regola d'oro: il refactoring va fatto su codice **coperto da test**; senza test non c'è certezza di non rompere nulla.

14.5 Dai diagrammi al codice: flusso di lavoro

